WEEK 1 · LESSON 2 OF 24 · HANDOUT

Reproducible Machine Learning

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=UY2WFPeEVN0 — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon DE 2022: Reproducible machine learning and science with Python — https://www.youtube.com/watch?v=0ShpadTHsro

🎓 **Go deeper:** Python Web Conf 2022: Using Reproducible Experiments to Create Better Machine Learning Models — https://www.youtube.com/watch?v=DY8LpArCuII

Your goal in this lesson: Make training repeatable with fixed seeds, configuration, pipelines, data fingerprints and pinned library versions, and prove it with a check.

Why reproducibility matters

Six months from now someone asks: "Why did the model refuse my loan? Which data and code made it?" If you cannot rebuild the exact same model, you cannot answer, debug or defend it. **Reproducible** means: same data + same code + same settings = same model.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

Hidden randomness

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]

def train_once(seed=None):
    tr, te = train_test_split(applicants, test_size=0.25, random_state=seed)
    forest = RandomForestClassifier(n_estimators=50, random_state=seed).fit(tr[FEATURES],
tr["repaid"])
    return round(roc_auc_score(te["repaid"], forest.predict_proba(te[FEATURES])[:, 1]), 4)

print("no seed:   ", [train_once() for _ in range(3)])
print("seed = 42: ", [train_once(42) for _ in range(3)])
```

Without a seed, every run gives a different score — so you cannot tell whether a change helped or you were just lucky. With a fixed seed, the runs are identical.

Put every setting in one config

```
import hashlib
import json

CONFIG = {
    "data": {"test_size": 0.25, "split_seed": 0},
    "features": FEATURES,
    "target": "repaid",
    "model": {"type": "logistic_regression", "C": 1.0, "max_iter": 1000},
    "threshold": 0.5,
}

def config_id(config):
    return hashlib.sha256(json.dumps(config, sort_keys=True).encode()).hexdigest()[:10]

print("config id:", config_id(CONFIG))
```

The **config id** is a short fingerprint: if any setting changes, the id changes.

A pipeline keeps preprocessing with the model

If you scale the data in one cell and train in another, it is easy to forget the scaling at serving time. A scikit-learn **Pipeline** stores both together, so the saved file always does the same steps:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

def train(config, data):
    tr, te = train_test_split(data, test_size=config["data"]["test_size"],
                              random_state=config["data"]["split_seed"],
stratify=data[config["target"]])
    pipe = Pipeline([("scale", StandardScaler()),
                     ("model", LogisticRegression(C=config["model"]["C"],
max_iter=config["model"]["max_iter"]))])
    pipe.fit(tr[config["features"]], tr[config["target"]])
    p = pipe.predict_proba(te[config["features"]])[:, 1]
    metrics = {"auc": round(roc_auc_score(te[config["target"]], p), 4),
               "accuracy": round(accuracy_score(te[config["target"]], p >=
config["threshold"]), 4)}
    return pipe, metrics

_, first = train(CONFIG, applicants)
_, second = train(CONFIG, applicants)
print(first, second)
assert first == second, "training is not reproducible!"
print("reproducible: the same config and data give the same model")
```

That `assert` is a tiny **reproducibility test**. Put it in your test suite (Lesson 4).

Fingerprint the data

```
def data_fingerprint(df):
    return hashlib.sha256(pd.util.hash_pandas_object(df,
index=False).values.tobytes()).hexdigest()[:16]

print("original data:", data_fingerprint(applicants))
changed = applicants.copy()
changed.loc[0, "income_rwf"] += 1000          # change ONE number
print("one value changed:", data_fingerprint(changed))
```

One changed value gives a completely different fingerprint. Store the fingerprint with every model — and keep the data itself in a safe, versioned place (a dated folder, object storage, or tools such as DVC).

Pin your library versions

```
import importlib.metadata as md

packages = ["scikit-learn", "pandas", "numpy", "joblib"]
pinned = [f"{p}=={md.version(p)}" for p in packages]
with open("requirements.txt", "w") as f:
    f.write("\n".join(pinned) + "\n")
print(open("requirements.txt").read())
```

`requirements.txt` with exact versions (`==`) lets anyone install the same libraries later.

A tidy folder layout

```
loan-model/
  data/            raw and processed data (or links to where it is stored)
  src/             train.py, features.py, serve.py
  tests/           test_data.py, test_model.py, test_api.py
  configs/         train.json
  models/          saved models + metadata (or a model registry)
  requirements.txt
  README.md        how to train, test and run
```

Key terms

- **Reproducible** — the same inputs always give the same model and results.
- **Random seed** — a number that fixes "random" choices.
- **Config** — one place that holds every training setting.
- **Pipeline** — preprocessing and model saved together.
- **Data fingerprint** — a hash that identifies the exact data.
- **Pinned versions** — exact library versions in requirements.txt.

Remember this: Fix seeds, keep settings in a config, use pipelines, fingerprint the data and pin library versions — then prove reproducibility with a test.

Name: _____ Date: _____

Activity

Task 1

Change `C` to 0.1 in `CONFIG`. Does the config id change? Does the AUC change?

Task 2

Remove `stratify` from `train`. Is it still reproducible?

Task 3

Write a function `save_run(config, metrics, data)` that stores the config, metrics and data fingerprint in one JSON file.

Task 4

Create the folder layout above for one of your own projects.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does reproducible mean in ML?

   A. Same data, code and settings give the same model and results
   B. The model is accurate
   C. The code is short
   D. The model is fast

2. Why fix a random seed?

   A. So results do not change from run to run and comparisons are fair
   B. To save memory
   C. To make the model better
   D. It hides errors

3. Why use a scikit-learn Pipeline?

   A. It is faster
   B. It tunes the threshold
   C. It removes the need for data
   D. Preprocessing and model are saved together, so serving does the same steps

4. What happens to a data fingerprint when one value changes?

   A. Nothing
   B. It becomes completely different
   C. It is deleted
   D. It increases by one

5. What does requirements.txt with == versions do?

   A. Lets anyone install exactly the same library versions later
   B. Installs the newest versions
   C. Removes old models
   D. Trains the model

6. Where should every training setting live?

   A. Spread across many cells
   B. Only in your memory
   C. In the model name
   D. In one config

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.